

一、 程序阅读 (20 分)

1. 深度优先生成树

题目分析：此题考察图的深度优先搜索 (DFS) 遍历及生成树的构建。

第一空：在访问顶点 v 后，需要获取 v 的第一条依附边。根据邻接表定义，应为 $p = G.vertices[v].firstarc;$ 。

第二空：DFS 递归进入更深一层的邻居。应为 $DFS_Tree(G, p->adjvex);$ 。

2. 共享栈 (Shared Stack)

题目分析：考察在一个数组中实现两个栈，两个栈底分别在数组两端，向中间生长。

Push 判满：当两个栈顶相邻时，栈满。即 $S.top[0] + 1 == S.top[1]$ 。

Push 操作：

栈 0 进栈 (从左往右)：先加指针，再存值。 $S.top[0]++;$ $S.data[S.top[0]] = x;$ (简写为 $S.data[++S.top[0]] = x;$)。根据题目填空风格，这里分为下标和赋值。

栈 1 进栈 (从右往左)：先减指针，再存值。 $S.top[1]--;$ $S.data[S.top[1]] = x;$ 。

Pop 操作：

栈 0 出栈： $x = S.data[S.top[0]--];$

栈 1 出栈： $x = S.data[S.top[1]++];$

答案：

$S.top[0] + 1 == S.top[1]$

++S.top[0]

--S.top[1]

S.top[0]--

S.top[1]++

3. 链表求交集 (Intersection of Lists)

题目分析：将两个递增有序链表 A 和 B 的交集存回 A, 要求空间复杂度 $O(1)$ (利用原节点)。

循环条件：两个链表都未遍历完时循环。pa && pb。

相等情况：pa->data == pb->data。保留 pa, 删除 pb。题目中 pc 均指向结果链表尾部。

pc->next = pa; pc = pa; (尾插法)

pa = pa->next;

temp = pb;

pb = pb->next;

free(temp); (释放 B 的节点)

A < B 情况：A 中元素小, 说明 A 的这个元素不在交集中 (因为是递增的), 删除 A 节点。

temp = pa; pa = pa->next; free(temp);

A > B 情况：B 中元素小, 删除 B 节点。

```
temp = pb; pb = pb->next; free(temp);
```

答案：

```
pa && pb
```

```
pc->next = pa;
```

```
pc->next = NULL;
```

4. 递归函数分析

题目分析：

代码有两个 riddle 函数版本。

版本 1 (Line 116)：是打印相邻值中较小的

版本 2 (Line 127)：递归求最小值

版本一对应的答案 23 16 16 34 34 29 29 时间复杂度是 $O(n)$

版本二没有打印语句，但是返回值为 16，时间复杂度是 $O(n)$

5. 平衡二叉树判断 (IsBalanced)

题目分析：后序遍历判断平衡性。

空树：深度为 0，返回 true。

递归检查：判断左右子树是否平衡。

```
IsBalanced(T->lchild, deepleft) && IsBalanced(T->rchild, deepright)
```

平衡条件：左右子树高度差绝对值 ≤ 1 。

```
abs(deepleft - deepright) <= 1 (或 dif >= -1 && dif <= 1)
```

答案：

$\text{IsBalanced}(T \rightarrow \text{lchild}, \text{deepleft}) \ \&\& \ \text{IsBalanced}(T \rightarrow \text{rchild}, \text{deerpight})$

$\text{abs}(\text{dif}) \leq 1$

二、 填空题 (10 分)

6、 后缀表达式求值：

表达式：a b c + * d -

还原为中缀表达式为 $a*(b+c)-d$

代入值：a=2, b=3, c=4, d=5

答案：9

7、

循环队列长度： $(\text{rear} - \text{front} + \text{MaxSize}) \% \text{MaxSize}$

数据：rear=0, front=3, MaxSize=6

删除一个元素,front 的值： $(3+1+6)\%6=4$

加入两个元素,rear 的值： $(0+2+6)\%6=2$

答案：3

8、

数组地址计算：

已知：A[0][0]地址 100，A[3][3]地址 220。假设按行优先存储。

设元素大小为 L ，每行元素个数为 N 。

$$Addr(3,3) = Addr(0,0) + (3 \times N + 3) \times L$$

$$220 = 100 + 3L(N + 1) \Rightarrow 120 = 3(N + 1) \Rightarrow 40 = (N + 1)$$

所以 $N=39$

$$\text{所以 } Addr(5,5) = Addr(0,0) + (5 \times 39 + 5) \times 1 = 100 + 200 = 300$$

○

9、

题目应该有问题，应该是进栈和出栈。 $T1=n$ 说明所有元素已经进栈，第 i 个出栈为 $n-i+1$ 。

10、

最少且连通，需要 $n+1$ 条边，构成环。

11、

一共 $2n$ 个指针，用了 $n-1$ ，还剩 $n+1$ 个空指针

○ 12、

在第 i 个位置后插入。若需从头查找第 i 个节点，时间复杂度为 $O(n)$ 。

○

13、

折半查找最大比较次数：

公式： $\lfloor \log_2 n \rfloor + 1$

计算： $\lfloor \log_2 125 \rfloor + 1 = 6 + 1 = 7$

答案：7

14、

题目不全，通常考察 Head() 取表头（元素），Tail() 取表尾（子表）。

15、

大根堆插入比较次数：

插入到堆底，然后向上调整。最坏情况是从叶子比较到根。

深度为 $\lfloor \log_2 n \rfloor$ 。

答案： $\lfloor \log_2 n \rfloor$

三、 综合应用题 (70 分)

16. 二叉树重建

已知：

这个我们之前说过快捷方法，横纵写一下中序和先序，可以容易地求出。

中序：B A D C F E G

先序：A B C D E F G



后序序列：B D F G E C A

三叉链表：在二叉链表基础上增加 parent 指针指向父节点。

对应森林：由 4 棵独立的树组成：(A-B), (C-D), (E-F), (G)。

○

17. 哈希表

题目：

Keys: 19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79

表长 16 (HT[0...15])

冲突解决：线性探测法

计算过程：

$19 \% 13 = 6 \rightarrow \text{HT}[6] = 19$

$14 \% 13 = 1 \rightarrow \text{HT}[1] = 14$

$23 \% 13 = 10 \rightarrow \text{HT}[10] = 23$

$1 \% 13 = 1$ (冲突, 探测+1) $\rightarrow \text{HT}[2] = 1$

$68 \% 13 = 3 \rightarrow \text{HT}[3] = 68$

$20 \% 13 = 7 \rightarrow \text{HT}[7] = 20$

$84 \% 13 = 6$ (冲突, 探测+1 $\rightarrow 7$ 冲突, +1 $\rightarrow 8$) $\rightarrow \text{HT}[8] = 84$

$27 \% 13 = 1$ (冲突 $\rightarrow 2$ 冲突 $\rightarrow 3$ 冲突 $\rightarrow 4$) $\rightarrow \text{HT}[4] = 27$

$55 \% 13 = 3$ (冲突 $\rightarrow 4$ 冲突 $\rightarrow 5$) $\rightarrow \text{HT}[5] = 55$

$11 \% 13 = 11$, 放 $\text{HT}[11]$

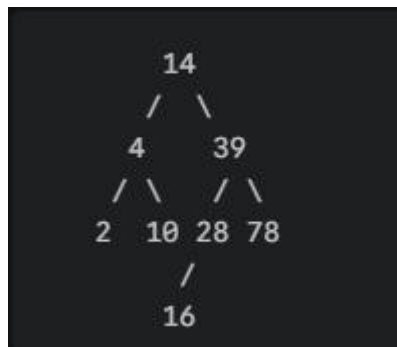
$10 \% 13 = 10$ (冲突, HT[10]已有 23) 探测 HT[11] (有 11), HT[12]\$(空), 放 HT[12]

$79 \% 13 = 1$ (冲突) 探测 2, 3, 4, 5, 6, 7, 8 (都有值) HT[9] (空), 放 HT[9]

18. 二叉排序树 (BST)

序列: 14, 4, 39, 28, 16, 2, 10, 78

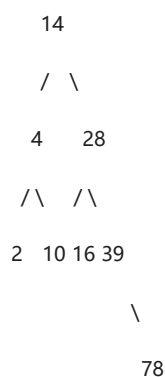
(1) 插入结果:



(2) 平均查找长度 (ASL):

$$ASL = (1*1 + 2*2 + 3*4 + 4*1) / 8 = (1 + 4 + 12 + 4) / 8 = 21 / 8 = 2.625$$

(3) 平衡调整 (AVL):



19. 图不清晰, 甚至不知道是单源点到其他节点还是结点之间。但是可以根据迪杰斯特拉算法或者弗洛伊德来计算。

20. 没有给图，做不了，但是按照之前讲的从前向后求最大，和从后往前求最小，可以方便得到事件的最早和最晚，再根据这个表格求活动的最早和最晚。

21. Dijkstra 最短路径

- 没给图，但是这个题目和 19 题思路很像，可能有一个题有问题，记重复了，应该不会一张卷子考两次的相同的算法。

22. 哈夫曼树

没给具体权值，但是哈夫曼树的题目一般不难。

23. 折半查找判定树

序列：2, 7, 8, 10, 24, 25, 28, 31, 33, 39, 41

(1) 画判定树：

Root: $\text{mid} = (1+11)/2 = 6 \rightarrow$ **25**

L-Child: $\text{range}[1,5], \text{mid}=3 \rightarrow$ **8**

R-Child: $\text{range}[7,11], \text{mid}=9 \rightarrow$ **33**

以此类推递归画出。

这里就不画了，word 写的不方便。

(2) 查找 28 的比较序列：

与 25 比较： $28 > 25$ ，向右。

与 33 比较： $28 < 33$ ，向左（区间 $[7,8]$ 即 28, 31）。

$\text{Range}[7,8], \text{mid}=(7+8)/2 = 7 \rightarrow$ **28**。

与 28 比较：相等，找到。

比较元素：25, 33, 28。

四、 算法分析题 (30 分)

24. 约瑟夫环 (Josephus Loop)

算法思路：使用循环链表模拟，或使用数组。这个题我多说一句，之前我讲冲刺班的时候压到这题了。

C++

```
// 模拟法，复杂度  $O(n*m)$ 
void Josephus(int n, int s, int m) {
    int p[n]; // 标记数组，0表示在圈内
    for(int i=0; i<n; i++) p[i]=0;

    int count = 0; // 出圈人数
    int current = s - 1; // 当前报数位置(数组下标)
    int say = 0; // 当前报的数

    while(count < n) {
        if(p[current] == 0) { // 如果人在圈内
            say++;
            if(say == m) {
                printf("%d ", current + 1); // 输出编号
                p[current] = 1; // 标记出圈
                count++;
                say = 0;
            }
        }
        current = (current + 1) % n; // 循环
    }
}
```

25. 根据先序和中序建立二叉链表

算法思路：递归划分。先序第一个是根，在中序中找到该根的位置，划分左右子树长度。

```
C++  
  
typedef struct BiTNode {  
    char data;  
    struct BiTNode *lchild, *rchild;  
} BiTNode, *BiTree;  
  
BiTree BuildTree(char *Pre, char *In, int len) {  
    if (len <= 0) return NULL;  
  
    BiTNode *root = (BiTNode*)malloc(sizeof(BiTNode));  
    root->data = *Pre;  
  
    // 在中序中找根的位置  
    int i;  
    for (i = 0; i < len; i++) {  
        if (In[i] == *Pre) break;  
    }  
  
    // 递归构建  
    root->lchild = BuildTree(Pre + 1, In, i);  
    root->rchild = BuildTree(Pre + 1 + i, In + i + 1, len - i - 1);  
  
    return root;  
}
```

26. 验证拓扑序列

算法思路：

遍历给定序列中的每一个顶点 u 。

对于 u ，检查图中所有指向 u 的顶点（即入边 $v \rightarrow u$ ）。

如果存在任意一个前驱节点 v ，在给定序列中出现在 u 的后面，则该序列不是拓扑序列。

或者使用入度表法：模拟拓扑排序过程，按序列顺序检查当前节点的入度是否为0。

```
C++  
  
bool IsTopologicalSort(ALGraph G, int *seq, int n) {  
    int indegree[MaxVertexNum] = {0};  
    // 1. 计算所有点入度  
    for (int i = 0; i < G.vexnum; i++) {  
        for (ArcNode *p = G.vertices[i].firstarc; p; p = p->nextarc) {  
            indegree[p->adjvex]++;  
        }  
    }  
  
    // 2. 按seq顺序检查  
    for (int i = 0; i < n; i++) {  
        int v = seq[i];  
        if (indegree[v] != 0) {  
            return false; // 当前点入度不为0，不能作为拓扑排序的下一个  
        }  
        // "删除"该点，减少邻接点入度  
        for (ArcNode *p = G.vertices[v].firstarc; p; p = p->nextarc) {  
            indegree[p->adjvex]--;  
        }  
    }  
    return true;  
}
```